
1. Initial Steps to Start the Project (JusticeServe)

Step 1: Understand the System Requirements

Read the modules and identify:

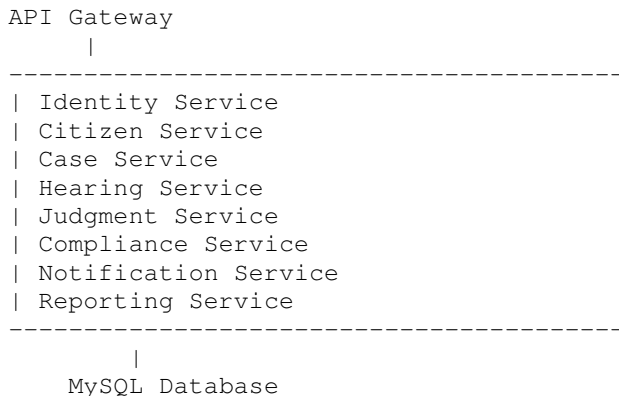
- Actors: Citizen, Lawyer, Judge, Clerk, Admin, Compliance Officer, Auditor
- Core workflows:
 - Citizen Registration
 - Case Filing
 - Hearing Scheduling
 - Judgment Delivery
 - Compliance Monitoring
 - Reporting

Create a **Use Case Diagram** and **System Architecture Diagram**.

Step 2: Define the System Architecture

Use a **Microservices Architecture** (as per your training).

Suggested services:



Frontend communicates with services using **REST APIs**.

Step 3: Setup Development Environment

Install the following:

Tool	Purpose
JDK 21	Java development
Maven	Build tool
MySQL	Database
Spring Boot	Backend framework

Tool	Purpose
Postman	API testing
Git	Version control
Angular 16	Frontend
Docker	Containerization
SonarQube	Code quality
VS Code / IntelliJ Development IDE	

Step 4: Create Git Repository

Create repository:

```
justice-serve-system
```

Structure:

```
justice-serve-system
|-- backend-services
|-- frontend-app
|-- database
|-- documentation
```

Step 5: Design Database Schema

Tables based on your document:

- User
- AuditLog
- Citizen
- CitizenDocument
- Case
- CaseDocument
- Hearing
- Proceeding
- Judgment
- CourtOrder
- ComplianceRecord
- Audit
- Report
- Notification

Use **MySQL Workbench** or **ER Diagram tools**.

Step 6: Setup Backend Spring Boot Project

Create project using **Spring Initializr**

Dependencies:

```
Spring Web
Spring Data JPA
```

MySQL Driver
Spring Security
Lombok
Spring Boot DevTools
Validation

Step 7: Create Backend Layers

Follow **clean architecture**:

controller
service
repository
entity
dto
config
exception

Example:

```
case-service
  controller
    CaseController
  service
    CaseService
  repository
    CaseRepository
  entity
    Case
  dto
    CaseRequest
    CaseResponse
```

Step 8: Implement REST APIs

Example:

```
POST /citizens
POST /cases
GET /cases/{id}
POST /hearings
POST /judgments
```

Test using **Postman**.

Step 9: Develop Frontend (Angular)

Pages:

Citizen Portal
Lawyer Dashboard
Judge Console
Clerk Panel
Admin Panel
Compliance Dashboard

Use:

- Angular
 - Bootstrap
 - REST API integration
-

Step 10: Integrate Backend + Frontend

Angular calls APIs:

```
http://localhost:8080/api/cases
```

Step 11: Add Logging and Monitoring

Use:

```
SLF4J  
Lombok  
SonarQube
```

Step 12: Containerize with Docker

Create:

```
Dockerfile  
docker-compose.yml
```

Step 13: Deploy

Deploy on:

```
Local server  
or  
Cloud (GCP)
```

2. Basic Tech Stack for the Project

Your **training stack** should be followed.

Layer	Technology
Frontend	Angular 16
UI	HTML5, CSS3, Bootstrap
Scripting	JavaScript
Backend	Java 21
Framework	Spring Boot 3
ORM	Spring Data JPA + Hibernate
API	RESTful APIs
Database	MySQL
Authentication	Spring Security

Layer	Technology
Build Tool	Maven
Version Control	Git
Testing	JUnit + Mockito
Logging	SLF4J + Lombok
Code Quality	SonarQube
Containerization	Docker
Cloud	GCP
CI/CD	GitHub / Git

3. Skills Required for This Project

Programming Skills

- Core Java
 - Object Oriented Programming
 - Java Collections
 - Exception Handling
-

Backend Development

- Spring Boot
 - REST API development
 - Microservices architecture
 - Spring Data JPA
 - Hibernate ORM
-

Database Skills

- SQL
 - Database design
 - Normalization
 - Query optimization
-

Frontend Development

- Angular
 - HTML5
 - CSS3
 - JavaScript
 - Bootstrap
-

DevOps Skills

- Git
 - Docker
 - CI/CD basics
 - Cloud deployment
-

Testing Skills

- JUnit
 - Mockito
 - API testing with Postman
-

Software Engineering Skills

- Agile methodology
 - SOLID principles
 - Debugging
 - Code quality
-

4. Technologies Used for Each Module

1 Identity & Access Management

Purpose: Authentication and authorization

Technologies:

- Spring Security
 - JWT Authentication
 - Spring Boot
 - MySQL
 - SLF4J Logging
-

2 Citizen Registration & Profile Management

Technologies:

- Spring Boot REST
- Spring Data JPA
- Hibernate
- MySQL
- Angular Forms

- Bootstrap UI
-

3 Case Filing & Document Management

Technologies:

- Spring Boot
 - File Upload APIs
 - JPA / Hibernate
 - MySQL
 - Angular File Upload
 - REST APIs
-

4 Hearing Scheduling & Court Proceedings

Technologies:

- Spring Boot
 - JPA / Hibernate
 - MySQL
 - Angular Calendar UI
 - REST APIs
-

5 Judgment & Order Management

Technologies:

- Spring Boot
 - Spring Data JPA
 - MySQL
 - Angular Dashboard
 - REST APIs
-

6 Compliance & Audit Management

Technologies:

- Spring Boot
 - Hibernate
 - MySQL
 - Logging with SLF4J
 - SonarQube auditing
-

7 Reporting & Analytics

Technologies:

- Spring Boot
- SQL Queries
- Angular Charts
- REST APIs

Libraries:

Chart.js
Angular Charts

8 Notifications & Alerts

Technologies:

- Spring Boot
 - Email Service (SMTP)
 - SMS Gateway API
 - Angular Notification UI
-

5. Suggested Microservice Structure

```
justice-serve
|
|-- identity-service
|-- citizen-service
|-- case-service
|-- hearing-service
|-- judgment-service
|-- compliance-service
|-- notification-service
|-- reporting-service
|
|-- api-gateway
|
|-- angular-frontend
```

6. Recommended Project Workflow (Best for Evaluation)

Follow this order:

- 1 Identity Service
- 2 Citizen Registration
- 3 Case Filing
- 4 Hearing Scheduling
- 5 Judgment Management

- 6 ☐ Compliance Monitoring
 - 7 ☐ Notifications
 - 8 Reporting Dashboard
 - 9 ☐ Angular Frontend
 - 10 Docker + Deployment
-